## Generics and Error Handling

### Task 1: Guided Activity – A Generic Stack

```
struct Stack<Element> {

    private var items: [Element] = []

    mutating func push(_ item: Element) {
        items.append(item)
    }
    mutating func pop() -> Element? {
        return items.popLast()
    }
    var top: Element? {
        return items.last
    }

    var isEmpty: Bool {
        return items.isEmpty
    }
    var count: Int {
        return items.count
    }
}
var intStack = Stack<Int>()

intStack.push(10)
intStack.push(20)
intStack.push(30)

print("Integer Stack:", intStack.count)
print("Top Element:", intStack.top ?? 0)
print("Popped:", intStack.pop() ?? 0)

var stringStack = Stack<String>()

stringStack.push("Apple")
stringStack.push("Banana")
stringStack.push("Orange")

print("String Stack:", stringStack.count)
print("Top Element:", stringStack.top ?? "Empty")
print("Popped:", stringStack.pop() ?? "Empty")


func largest<T: Comparable>(in items: [T]) -> T? {
    return items.max()
}

print("Largest Integer:", largest(in: [10, 50, 30]) ?? 0)

print("Largest String:", largest(in: ["Apple", "Orange", "Banana"]) ?? "")

func swapValues<T>(_ a: inout T, _ b: inout T) {
    let temp = a
    a = b
    b = temp
}
```

```
var x = 10
var y = 20

swapValues(&x, &y)

print("After Swap:", x, y)

var first = "Hello"
var second = "Swift"

swapValues(&first, &second)

print("After Swap:", first, second)
```

Output

```
Integer Stack: 3
Top Element: 30
Popped: 30
String Stack: 3
Top Element: Orange
Popped: Orange
Largest Integer: 50
Largest String: Orange
After Swap: 20 10
After Swap: Swift Hello
```

**Task 2: Practice Task – ATM Withdrawal with Errors**

```
import Foundation

enum ATMError: Error {
    case insufficientFunds(shortBy: Double)
    case invalidAmount
    case dailyLimitExceeded
}
struct Account {

    var balance: Double
    var dailyWithdrawn: Double = 0

    let dailyLimit: Double = 20000
    mutating func withdraw(_ amount: Double) throws {

        if amount <= 0 {
            throw ATMError.invalidAmount
        }
        if dailyWithdrawn + amount > dailyLimit {
            throw ATMError.dailyLimitExceeded
        }
        if amount > balance {
            throw ATMError.insufficientFunds(
```

```
                shortBy: amount - balance
            )
        }

        balance -= amount
        dailyWithdrawn += amount

        print("Withdrawal Successful!")
        print("Remaining Balance:", balance)
    }
}
var account = Account(balance: 10000)

print("\nTest 1: Withdraw 500")

do {
    try account.withdraw(500)
}
catch ATMError.insufficientFunds(let shortBy) {
    print("Insufficient Funds. Short by:", shortBy)
}
catch ATMError.invalidAmount {
    print("Invalid Amount")
}
catch ATMError.dailyLimitExceeded {
    print("Daily Limit Exceeded")
}
catch {
    print("Unknown Error")
}
print("\nTest 2: Withdraw -10")

do {
    try account.withdraw(-10)
}
catch ATMError.insufficientFunds(let shortBy) {
    print("Insufficient Funds. Short by:", shortBy)
}
catch ATMError.invalidAmount {
    print("Invalid Amount")
}
catch ATMError.dailyLimitExceeded {
    print("Daily Limit Exceeded")
}
catch {
    print("Unknown Error")
}
print("\nTest 3: Withdraw 25000")

do {
    try account.withdraw(25000)
}
catch ATMError.insufficientFunds(let shortBy) {
    print("Insufficient Funds. Short by:", shortBy)
}
catch ATMError.invalidAmount {
    print("Invalid Amount")
}
catch ATMError.dailyLimitExceeded {
    print("Daily Limit Exceeded")
}
```

```
catch {
    print("Unknown Error")
}
print("\nTest 4: Withdraw 15000")

do {
    try account.withdraw(15000)
}
catch ATMError.insufficientFunds(let shortBy) {
    print("Insufficient Funds. Short by:", shortBy)
}
catch ATMError.invalidAmount {
    print("Invalid Amount")
}
catch ATMError.dailyLimitExceeded {
    print("Daily Limit Exceeded")
}
catch {
    print("Unknown Error")
}

print("\nTest 5: Using try?")

let result: Void? = try? account.withdraw(100)

if result != nil {
    print("Withdrawal completed")
} else {
    print("Withdrawal failed")
}
```

Output

```
Test 1: Withdraw 500
Withdrawal Successful!
Remaining Balance: 9500.0

Test 2: Withdraw -10
Invalid Amount

Test 3: Withdraw 25000
Daily Limit Exceeded

Test 4: Withdraw 15000
Insufficient Funds. Short by: 5500.0

Test 5: Using try?
Withdrawal Successful!
Remaining Balance: 9400.0
```

### Task 3: Challenge – Result Type and Defer

```
enum AgeError: Error {
    case empty
    case notANumber
```

```
    case outOfRange
}

func parseAge(_ text: String) throws -> Int {


    if text.isEmpty {
        throw AgeError.empty
    }


    guard let age = Int(text) else {
        throw AgeError.notANumber
    }
    if age < 1 || age > 120 {
        throw AgeError.outOfRange
    }

    return age
}

let result = Result {
    try parseAge("25")
}

switch result {

case .success(let age):
    print("Valid Age:", age)

case .failure(let error):
    print("Error:", error)
}
let test1 = Result { try parseAge("") }
let test2 = Result { try parseAge("Hello") }
let test3 = Result { try parseAge("150") }
let test4 = Result { try parseAge("22") }

print(test1)
print(test2)
print(test3)
print(test4)

func saveReport() throws {

    print("Opening file")

    defer {
        print("Closing file")
    }

    print("Saving report")

    throw AgeError.notANumber
}

do {
    try saveReport()
}
catch {
    print("Error while saving report:", error)
```

```
}
```

Output

```
Valid Age: 25
failure(main.AgeError.empty)
failure(main.AgeError.notANumber)
failure(main.AgeError.outOfRange)
success(22)
Opening file
Saving report
Closing file
Error while saving report: notANumber
```